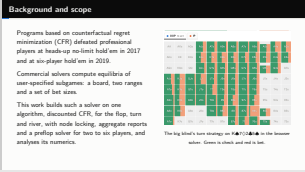


Timing for 60 minutes. Problem and equilibria 7 · CFR and DCFR 7 · Traversal and leaves 9 · Cost, memory, precision and parallelism 9 · Convergence and validation 6 · Demonstration 9 · Extensions and next steps 8 · Questions 5.

Opening line: "A poker solver is one numerical method, discounted counterfactual regret minimization, applied to a structured bilinear saddle-point problem. This talk is about what makes that method fast and accurate: how an iteration is organized, what it costs, how precise it must be and how it is split across processors."

Every measured number comes from `web/experiments/run_all.sh` and `tex/make_talk_data.py`. The machine is a two-core cloud container, so absolute times are pessimistic.



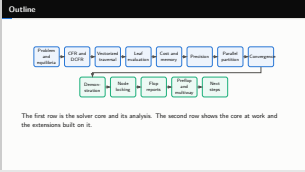
Libratus and DeepStack (2017) and Pluribus (2019) are the milestones, and their inner loop is a member of the CFR family. The feature list of this project follows the commercial tools PioSolver and MonkerSolver: flop, turn and river solving, arbitrary bet trees, rake, node locking, aggregate flop reports, preflop and multiway solving.

On the algorithm side the scope is narrow on purpose: one method and its numerical analysis. New algorithms are on the next-steps slide.

Range-Vectorized Discounted Regret Minimization for No-Limit Hold'em: Complexity, Precision and Parallelism

└Outline

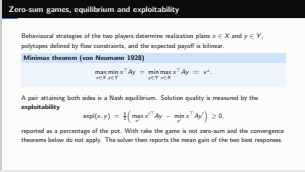
Keep this slide short.





The running example is the most common spot in online poker: the button opens, the big blind calls, and the flop is K♠ 7♢ 2♣. The big blind has 488 hands and the button 460 that avoid the board.

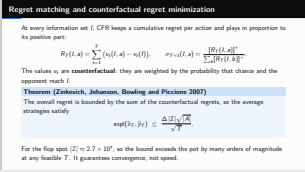
The table gives the spots of the convergence study, one tree per street. A river spot has a few dozen decision nodes; a flop spot with one bet size per street has tens of thousands, because every turn and river card opens a new copy of the later streets.



This is a convex-concave saddle-point problem with bilinear coupling, and the exploitability is the primal-dual gap. Commercial solvers stop between 0.1% and 0.5% of the pot.

The sequence-form linear program of Koller, Megiddo and von Stengel solves the game exactly, but its matrix has a dense hand-by-hand block at every leaf; a flop tree has about seventy thousand leaves with blocks of size about 500×500. Iterative methods that only need products with A and $A^{\top}$ are the only option, and those products never need A either.

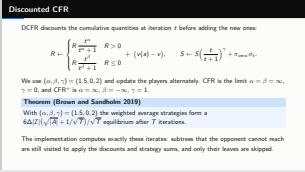
With rake, DCFR is used as a heuristic; the measured exploitability is still reported and in our runs it decreases in the same way.



Regret matching (Hart and Mas-Colell 2000) keeps the maximal regret of order $\sqrt{T}$ by Blackwell approachability: the instantaneous regret is orthogonal to the current positive regret vector, so the squared norm of the positive regrets grows at most linearly.

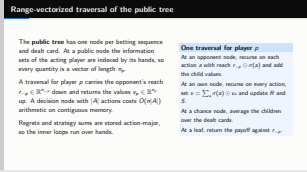
CFR applies regret matching at each information set with the counterfactual value as payoff. The decomposition theorem bounds the global regret by the sum of these local regrets, and average regret ε for both players gives an ε-equilibrium of the averages in our normalization.

Δ is the payoff range, $|\mathcal{I}|$ the number of information sets (hands times decision nodes) and $|A|$ the largest number of actions. With $\Delta \approx 2 \cdot 97.5$ chips, $|\mathcal{I}| \approx 2.7 \times 10^7$ and $|A| = 3$, the bound at $T = 10^4$ is above 10^7 chips. The observed behaviour is far better and is measured later.



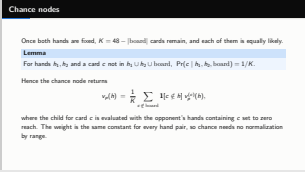
Discounting is a forgetting factor. Positive regrets lose a fraction $1/(t^{1.5}+1)$, negative regrets are halved, and the average gives weight proportional to t^2 to iteration t. Alternating updates are Gauss–Seidel in place of Jacobi.

Exactness matters. An earlier version skipped subtrees with zero opponent reach entirely, which also skipped their discounts and strategy-sum updates. On a small flop game this left the exploitability at 0.64 chips after 1000 iterations instead of 0.0025, a factor of 250. A test now checks that the regrets and sums agree bit for bit with a traversal that never skips. The extra cost is the decision work of the skipped subtrees; their leaves are still skipped.



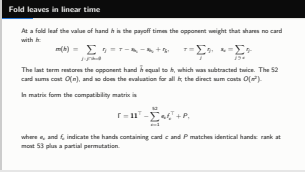
A naive implementation traverses the tree once per hand pair. The vectorized form traverses it once per player and iteration, with vector arithmetic of length about 500 at each node.

The products Ay and $A^\top x$ of the earlier slide are exactly the values returned to the root. The opponent's reach plays the role of y, and the payoff blocks are applied at the leaves without being stored. All buffers for one depth are allocated once, so a traversal allocates no memory.



The conditional probability of the next card depends only on the four known private cards, which are excluded by the indicator and by zeroing the opponent's reach. Using $1/(52 - |\text{board}|)$ would bias every value.

The lemma is checked against exact linear programs with explicit chance nodes, one chance level on the turn and two on the flop (validation slide).


Fold leaves in linear time

At a fold leaf the value of hand h is the payoff times the opponent weight that shares no card with h:

$$m(h) = \sum_{j:\, j^{\mathrm{rank}} \cap h = \emptyset} r_j = \tau - a_{h^{\mathrm{rank}}} - a_{h^{\mathrm{rank}}} + \tau_{h^{\mathrm{rank}}}, \qquad \tau = \sum_j r_j, \quad a_v = \sum_{j \ni v} r_j.$$

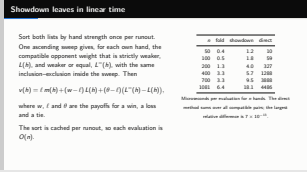
The last term restores the opponent hand $\hat{h}$ equal to h, which was subtracted twice. The 52 card sums cost $O(n)$, and so does the evaluation for all h; the direct sum costs $O(n^2)$.

In matrix form the compatibility matrix is

$$\Gamma = \mathbf{1}\mathbf{1}^{\top} - \sum_{c=1}^{52} e_c e_c^{\top} + P,$$

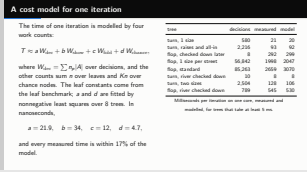
where e_c and ℓ_c indicate the hands containing card c and P matches identical hands: rank at most 53 plus a partial permutation.

Inclusion–exclusion over the two cards of h. Two different two-card hands share at most one card, except the identical hand, which shares both and is subtracted twice. This is the card-removal effect that players talk about, computed exactly in one pass over each list (Johanson and coauthors 2011).



After sorting, the sign matrix of the showdown is lower triangular ones minus upper triangular ones with tie blocks on the diagonal: a semiseparable matrix, so its product with a vector is a prefix sum. The compatibility mask is handled by per-card sums updated inside the sweep.

At 1081 hands the sweep is several hundred times faster than the direct sum and agrees to rounding. A flop iteration evaluates about 42,336 showdown leaves per traversal.



A cost model for one iteration

The time of one iteration is modelled by four work counts:

$$T \approx a\,W_{\text{dive}} + b\,W_{\text{down}} + c\,W_{\text{fold}} + d\,W_{\text{chance}},$$

where $W_{\text{dive}} = \sum_n n_a |A|$ over decisions, and the other counts sum n over leaves and Kn over chance nodes. The leaf constants come from the leaf benchmark; a and d are fitted by nonnegative least squares over 8 trees. In nanoseconds,

$$a = 21.9, \quad b = 34, \quad c = 12, \quad d = 4.7,$$

and every measured time is within 17% of the model.

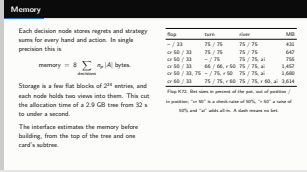
tree	decisions	measured	model
turn, 1 size	580	21	20
turn, ruiset and all-in	2,216	93	92
flop, checked down later	8	262	299
flop, 1 size per street	56,842	1968	2047
flop, standard	85,263	2959	3070
turn, river checked down	10	8	8
turn, two sizes	2,504	116	106
flop, river checked down	789	545	530

Milliseconds per iteration on one core, measured and modelled, for trees that take at least 5 ms.

Fitting all four constants at once is ill-posed: the fold work is nearly proportional to the decision work in every tree, so the columns are almost collinear. Fixing the leaf constants from the separate benchmark removes the collinearity.

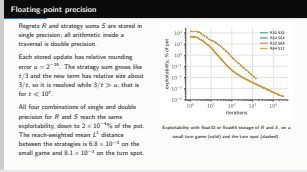
On the flop tree the leaves take about 42% of the time and the decisions most of the rest. For comparison, the same JavaScript runtime runs the bare DCFR update loop at 3.3 ns per entry and copies float32 arrays at 5.4 GB/s, so a is about 7 times the bare update. The gap is traversal overhead (regret matching, reach products, recursion), not memory bandwidth.

Adding a bet size multiplies the tree, because it adds a branch at every node of the street and a copy of all later streets below it.



Memory is the binding constraint in a browser. One typed array per node made the allocator the bottleneck; flat blocks make allocation and garbage collection cheap and keep each node's data contiguous.

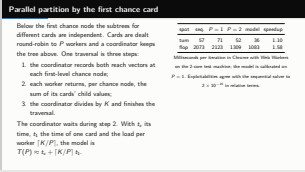
Below a chance node the subtrees for different cards have the same shape and differ only in the hands, so one card's subtree times the number of cards is an accurate estimate.



The strategy is a ratio of sums, so relative errors of the sums enter the strategy as relative errors of size u per update. A term is lost only when its relative size falls below u, which needs about 10^7 iterations.

The regrets grow like $\sqrt{t}$ and are resolved for longer. Rounding differences are amplified by the iteration, so the curves are not identical, but they cross repeatedly and differ by much less than the exploitability itself. An earlier version of the solver seemed to show a single-precision floor near 10^{-3}% of the pot; it disappeared with the exact zero-reach handling of the DCFR slide, so it was an artefact of the iteration, not of rounding.

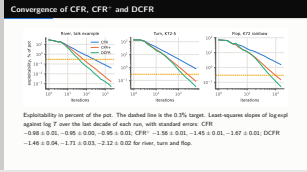
Half the memory for the same result. This is why commercial solvers can go further and compress to 16-bit integers.



This is a domain decomposition of the tree with the first-level chance nodes as interface. The messages are two vectors of length n per chance node and worker, at most 44 kB per traversal, against millions of operations per card. The coordinator's share is under a millisecond.

The results differ from the sequential solver only in the order in which the chance sums are added; they are not bitwise identical. On the flop, two workers reach 83% of the model's prediction. The model ignores message latency and the competition of the two workers for memory bandwidth and cache, and the main thread also runs the page. The model predicts 304 ms with eight workers. On the turn the passes are short, so the latency matters more.

In Node's worker threads the same partitioned traversal runs about 48% slower than on the main thread for reasons we have not identified; in Chrome the overhead with one worker is about 2%. In the browser the workers are Web Workers and the coordinator is the main thread. Games without a chance node, river spots, run in one worker.



All three methods use the same code and differ only in (α, β, γ). DCFR is fastest on every street. The slopes are empirical rates over one decade of iterations and should not be read as asymptotic rates; the worst-case bound is $T^{-1/2}$ for all three. Explaining why the modern variants do so much better on poker trees is an open problem.

Iterations to a fixed accuracy

DCFR on five river, five turn and three flop spots, with one tree per street. The table gives the median and the range of the iterations needed to reach each accuracy, in percent of the pot.

street	spots	1%	0.3%	0.1%
river	5	46 (46–53)	95 (82–110)	170 (147–228)
turn	5	95 (82–95)	170 (147–198)	299 (299–344)
flop	3	40 (34–40)	71 (61–71)	110 (110–127)

The iterations for a practical accuracy vary by less than a factor of two between spots and do not grow with the size of the tree. The running time to a fixed accuracy is therefore governed by the cost per iteration, which grows by three orders of magnitude from the river to the flop.

Boards: river K72T4, A8492 two-tone, T9832, QJ552, 762KA; the turn spots are the first four cards of the same boards; flop spots K72 rainbow, T98 two-tone and A84 two-tone. Ranges are the big blind's call and the button's open.

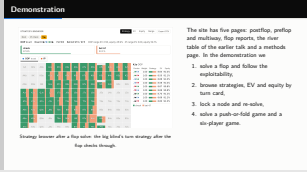
This is the practical statement behind "a few hundred iterations": it is a measured median over spots, not a theorem.

Validation

test	reference	result
River, talk example	the earlier independent solver	exploitability 1.34×10^{-4} and 1.34×10^{-3} chips after 5000 iterations
Turn, one chance level	exact LP, 14 and 12 hands	value 24.92419862 against 24.92418856
Flop, two chance levels	exact LP, 6 and 5 hands	value 27.139343 against 27.139333
Heads-up push or fold	LP on the same tables	values agree to five digits
Parallel partition	sequential solver	values agree to 10^{-14}
Zero-reach shortcut	traversal without shortcut	regrets and sums bit for bit
Leaf evaluation	direct $O(n^2)$ sums	agreement to 10^{-12}

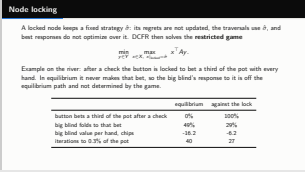
Each component has an independent check. The two LP tests build the game separately in Python with explicit chance nodes and solve the sequence-form LP exactly, so they test chance weighting, card removal and showdowns together; the remaining difference equals the exploitability left after 20,000 or 10,000 DCFR iterations. Values are compared rather than strategies, because equilibria are not unique.

A comparison with an established open-source solver on the same flop tree is the natural next test.



Switch to the browser. The script is app/DEMO_NOTES.md; budget nine minutes.

(1) Postflop page, example spot K72, one size per street. Build and solve; point at the memory estimate, the worker count and the exploitability curve with the 0.3% target. (2) The big blind has no flop bet in this tree, so the first decision is the button's: click check, then the five of spades. Show the grid, then EV and equity, and point at one square. (3) Node locking: edit, choose the bet, paint a row, lock, continue. (4) Preflop page, heads-up push or fold at 10 bb: 58% shove and 37% call. Then the six-max preset and the first-in table.



Node locking

A locked node keeps a fixed strategy $\hat{\sigma}$: its regrets are not updated, the traversals use $\hat{\sigma}$, and best responses do not optimize over it. DCFR then solves the **restricted game**

$$\min_{x \in \mathcal{X}} \max_{y \in \mathcal{Y},\, y_{\text{locked}} = \hat{\sigma}} x^\top A y.$$

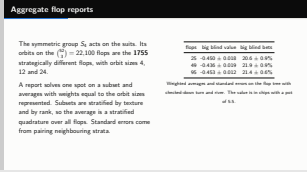
Example on the river: after a check the button is locked to bet a third of the pot with every hand. In equilibrium it never makes that bet, so the big blind's response to it is off the equilibrium path and not determined by the game.

	equilibrium	against the lock
button bets a third of the pot after a check	0%	100%
big blind folds to that bet	46%	29%
big blind value per hand, chips	-16.2	-6.2
iterations to 0.3% of the pot	40	27

Locking answers the practical question: what is the best response to a known mistake, while every other node also adapts.

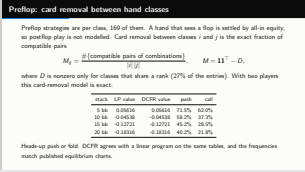
Numerically, the restricted game is no harder: solved from scratch, it reaches 0.3% of the pot in about as many iterations as the free game.

On flop spots a node below a chance card is locked for that card only, which matches commercial tools.



Burnside's lemma gives 1755 (appendix). Orbit sizes are 24 for rainbow boards, 12 for two-tone and paired boards and 4 for monotone and trips boards.

The report is a quadrature of a function of the flop. The collapsed-strata variance estimator pairs neighbouring strata and treats the pair as one stratum with two draws, which overestimates the variance slightly. The estimates agree within their standard errors as the subset grows.


Preflop: card removal between hand classes

Preflop strategies are per class, 169 of them. A hand that sees a flop is settled by all-in equity, so postflop play is not modelled. Card removal between classes i and j is the exact fraction of compatible pairs

$$M_{ij} = \frac{\#\{\text{compatible pairs of combinations}\}}{|i|\,|j|}, \qquad M = \mathbf{1}\mathbf{1}^{\top} - D,$$

where D is nonzero only for classes that share a rank (27% of the entries). With two players this card-removal model is exact.

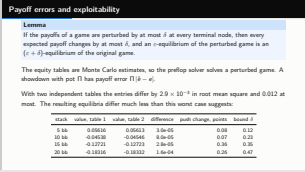
stack	LP value	DCFR value	push	call
5 bb	0.06646	0.06616	71.5%	62.0%
10 bb	-0.04538	-0.04538	58.2%	37.3%
15 bb	-0.12721	-0.12721	45.2%	28.5%
20 bb	-0.18316	-0.18316	40.2%	21.8%

Heads-up push or fold. DCFR agrees with a linear program on the same tables, and the frequencies match published equilibrium charts.

The equity tables are computed once offline: 169 by 169 class equities by Monte Carlo with 50,000 deals per pair, and exact pair counts.

The product Mr costs 169^2 dense, but with D sparse it is a sum minus a sparse correction, about three times cheaper. Products are cached per reach vector, because a vector formed at one node is shared by every leaf below it.

The LP has 169 strategy variables and 169 constraints. At 10 bb the frequencies are the published Nash push-or-fold numbers.


Payoff errors and exploitability

Lemma

If the payoffs of a game are perturbed by at most δ at every terminal node, then every expected payoff changes by at most δ, and an ε-equilibrium of the perturbed game is an $(\varepsilon + \delta)$-equilibrium of the original game.

The equity tables are Monte Carlo estimates, so the preflop solver solves a perturbed game. A showdown with pot Π has payoff error $\Pi|\hat{a} - a|$.

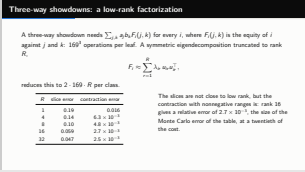
With two independent tables the entries differ by 2.9×10^{-3} in root mean square and 0.012 at most. The resulting equilibria differ much less than this worst case suggests:

stack	value, table 1	value, table 2	difference	push change, points	bound δ
5 bb	0.05616	0.05613	3.0e-05	0.08	0.12
10 bb	-0.04538	-0.04546	8.0e-05	0.07	0.23
15 bb	-0.12721	-0.12723	2.8e-05	0.36	0.35
20 bb	-0.18316	-0.18332	1.6e-04	0.26	0.47

Proof of the lemma: each best-response value is a maximum of expected payoffs, each of which moves by at most δ, so each half of the exploitability moves by at most $\delta/2$ in our normalization.

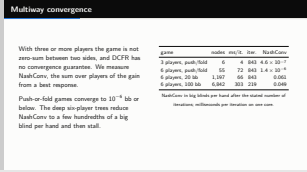
The worst-case bound uses the largest table error and is pessimistic: errors of different entries have different signs and average out over a range. The two-table experiment measures the actual effect.

The same lemma covers the low-rank approximation of the next slide, with δ equal to the pot times the contraction error.

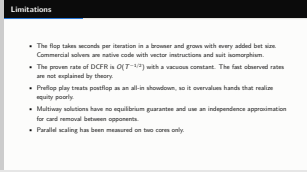


Errors are the largest over the 169 classes. The slice error is the relative Frobenius error of F_i; the contraction error is the largest relative error of $a^\top F_i b$ over 20 pairs of random ranges per class, with entries uniform on $[0, 1]$ and each kept with probability 0.4.

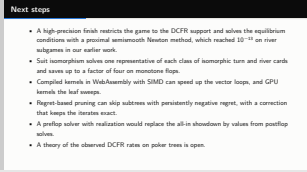
The tensor has 4.8 million entries, estimated with 1000 deals per sorted triple.



This is consistent with the literature: Pluribus used CFR-type self-play for six players without a guarantee, and it worked well in practice. With three players in a pot, opponents' cards are treated as independent given the hero's hand, which ignores card removal between the opponents.

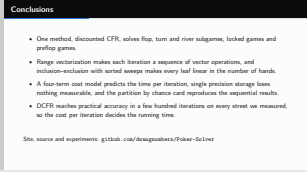


The browser engine is plain JavaScript; the next slide lists the steps that close most of the gap to native solvers. The preflop limitation is the largest modelling simplification: MonkerSolver solves the postflop as well, and simpler tools use realization factors.



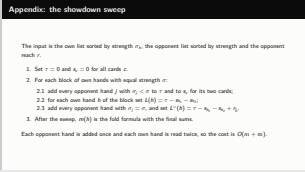
The first item is the research continuation (`talk-newton.pdf`): from a DCFR start, a proximal Newton method with interior-point crossover finished small and medium river games to machine precision. Extending it to turn and flop trees needs the structured linear algebra of the leaves inside the Newton systems.

The pruning item connects to the exactness slide: naive pruning changed the iterates and slowed convergence by a large factor, so a correct version must apply the skipped discounts lazily.



Return to the opening: the solver is a few lines of arithmetic, and its speed comes from structure exploited at the level of vectors, leaves, memory and processors. Then take questions.

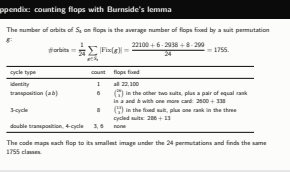
The identical hand $\bar{h}$ cannot be strictly weaker than h, so the correction appears only in $L^{=}$.


Appendix: the showdown sweep

The input is the own list sorted by strength σ_h, the opponent list sorted by strength and the opponent reach π.

1. Set $\tau \sqsubset 0$ and $s_c \sqsubset 0$ for all cards c.
2. For each block of own hands with equal strength σ:
   2.1 add every opponent hand j with $\sigma_j < \sigma$ to τ and to s_c for its two cards;
   2.2 for each own hand h of the block set $L(h) \sqsubset \tau - m_c - m_{\bar{h}}$;
   2.3 add every opponent hand with $\sigma_j = \sigma$, and set $L^{=}(h) \sqsubset \tau - s_{c_1} - s_{c_2} + \pi_j$.
3. After the sweep, $m(h)$ is the fold formula with the final sums.

Each opponent hand is added once and each own hand is read twice, so the cost is $O(n + m)$.



Appendix: counting flops with Burnside's lemma

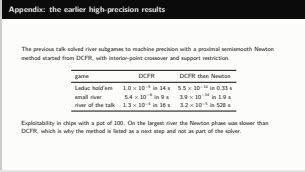
The number of orbits of S_4 on flops is the average number of flops fixed by a suit permutation g:

$$\#\text{orbits} = \frac{1}{|S_4|}\sum_{g\in S_4}|\mathrm{Fix}(g)| = \frac{22100 + 6\cdot 2938 + 8\cdot 299}{24} = 1755.$$

cycle type	count	flops fixed
identity	1	all $22\,100$
transposition $(a\,b)$	6	$\binom{13}{2}$ in the other two suits, plus a pair of equal rank in a and b with one more card: $2600 + 338$
3-cycle	8	$\binom{13}{3}$ in the fixed suit, plus one rank in the three cycled suits: $286 + 13$
double transposition, 4-cycle	3, 6	none

The code maps each flop to its smallest image under the 24 permutations and finds the same 1755 classes.

A flop has three cards, so a permutation that moves every suit cannot fix it unless its cards are permuted among themselves, which needs a cycle of length dividing the number of cards in the cycled suits. That rules out double transpositions and 4-cycles.


Appendix: the earlier high-precision results

The previous talk solved river subgames to machine precision with a proximal semismooth Newton method started from DCFR, with interior-point crossover and support restriction.

game	DCFR	DCFR then Newton
Leduc hold'em	1.0×10^{-4} in 14 s	5.5×10^{-16} in 0.33 s
small river	5.4×10^{-4} in 9 s	3.9×10^{-16} in 1.9 s
river of the talk	1.3×10^{-4} in 16 s	3.2×10^{-16} in 528 s

Exploitability in chips with a pot of 100. On the largest river the Newton phase was slower than DCFR, which is why the method is listed as a next step and not as part of the solver.

These are the numbers from `talk-newton.pdf`. The honest summary is that the finish is spectacular on small and medium games and not yet competitive on the largest river; the linear algebra of the dense leaf blocks inside the Newton systems is the bottleneck.